title: peppermint
section: DevOps
category: Organization
type: taxonomy
level: 3
status: active
up: Peppermint
created: 2026-04-7
updated: 2026-04-7
reviewed: 2026-04-7
tags: [DevOps,Peppermint]

Examples api usage
Examples api advanced usage
Ticket api usage
Ticket state change

Peppermint API Documentation

Overview

Peppermint is a ticketing system built with Fastify (Node.js/TypeScript) backend, PostgreSQL database, and Next.js frontend. The API runs on port `5003` and provides comprehensive ticket management, user authentication, client management, and webhook integrations.

Base Configuration

Database Connection

- Host: `peppermint-db`
- Port: `5433` (external) / `5432` (internal)
- User: `peppermint`
- Password: `1234`
- Database: `peppermint`

Environment Variables

- `DB_USERNAME` : peppermint

- `DB_PASSWORD` : 1234
- `DB_HOST` : peppermint-db
- `SECRET` : peppermint4life (base64 encoded JWT secret)

Authentication Method

- JWT tokens issued with 8-hour expiration
- Session-based authentication with session storage in PostgreSQL
- Bearer token format: `Authorization: Bearer <token>`

Authentication Endpoints

Register User (Admin Only)

```
POST /api/v1/auth/user/register
```

Requires: Admin access

Request Body:

```
{
  "email": "string",
  "password": "string",
  "name": "string",
  "admin": true
}
```

Response:

```
{
  "success": true
}
```

Register External User

```
POST /api/v1/auth/user/register/external
```

Request Body:

```
{
  "email": "string",
  "password": "string",
  "name": "string",
  "language": "en"
}
```

Response:

```
{
  "success": true
}
```

Login (Password)

```
POST /api/v1/auth/login
```

Request Body:

```
{
  "email": "string",
  "password": "string"
}
```

Response:

```
{
  "token": "jwt_token_string",
  "user": {
    "id": "uuid",
    "email": "string",
    "name": "string",
    "isAdmin": false,
    "language": "en",
    "ticket_created": true,
    "ticket_status_changed": true,
    "ticket_comments": true,
    "ticket_assigned": true,
```

```
    "firstLogin": true,
    "external_user": false
  }
}
```

Check Authentication Method

```
GET /api/v1/auth/check
```

Response:

```
{
  "success": true,
  "message": "SSO not enabled",
  "oauth": false
}
```

Or if SSO enabled:

```
{
  "type": "oidc",
  "success": true,
  "url": "authorization_url"
}
```

Get User Profile

```
GET /api/v1/auth/profile
```

Headers: `Authorization: Bearer <token>`

Response:

```
{
  "user": {
    "id": "uuid",
    "email": "string",
    "name": "string",
```

```
    "isAdmin": false,
    "language": "en",
    "ticket_created": true,
    "ticket_status_changed": true,
    "ticket_comments": true,
    "ticket_assigned": true,
    "sso_status": false,
    "version": "string",
    "notifications": [],
    "external_user": false
  }
}
```

Reset Password (User)

```
POST /api/v1/auth/reset-password
```

Request Body:

```
{
  "password": "new_password"
}
```

Response:

```
{
  "success": true
}
```

Reset Password (Admin)

```
POST /api/v1/auth/admin/reset-password
```

Requires: `user::manage` permission

Request Body:

```
{
  "password": "new_password",
  "user": "user_id"
}
```

Response:

```
{
  "success": true
}
```

Forgot Password

```
POST /api/v1/auth/password-reset
```

Request Body:

```
{
  "email": "string",
  "link": "reset_link_url"
}
```

Response:

```
{
  "success": true
}
```

Verify Reset Code

```
POST /api/v1/auth/password-reset/code
```

Request Body:

```
{
  "code": "string",
```

```
  "uuid": "string"
}
```

Response:

```
{
  "success": true
}
```

Reset Password with Code

```
POST /api/v1/auth/password-reset/password
```

Request Body:

```
{
  "password": "new_password",
  "code": "string"
}
```

Response:

```
{
  "success": true
}
```

Update Profile

```
PUT /api/v1/auth/profile
```

Requires: `user::update` permission

Request Body:

```
{
  "name": "string",
  "email": "string",
```

```
  "language": "en"
}
```

Response:

```
{
  "user": {}
}
```

Update Email Notification Settings

```
PUT /api/v1/auth/profile/notifcations/emails
```

Requires: `user::update` permission

Request Body:

```
{
  "notify_ticket_created": true,
  "notify_ticket_assigned": true,
  "notify_ticket_comments": true,
  "notify_ticket_status_changed": true
}
```

Response:

```
{
  "user": {}
}
```

Logout User

```
GET /api/v1/auth/user/:id/logout
```

Response:

```
{
  "success": true
}
```

Delete User

```
DELETE /api/v1/auth/user/:id
```

Requires: `user::delete` permission

Response:

```
{
  "success": true
}
```

User Management Endpoints

Get All Users

```
GET /api/v1/users/all
```

Requires: `user::read` permission

Response:

```
{
  "users": [
    {
      "id": "uuid",
      "name": "string",
      "email": "string",
      "isAdmin": false,
      "createdAt": "datetime",
      "updatedAt": "datetime",
      "language": "en"
    }
  ],
```

```
  "success": true
}
```

Create User (Admin)

```
POST /api/v1/user/new
```

Request Body:

```
{
  "email": "string",
  "password": "string",
  "name": "string",
  "admin": false
}
```

Response:

```
{
  "success": true
}
```

Admin Reset User Password

```
PUT /api/v1/user/reset-password
```

Request Body:

```
{
  "password": "new_password",
  "id": "user_id"
}
```

Response:

```
{
  "message": "password updated success",
```

```
  "failed": false
}
```

Mark Notification as Read

```
GET /api/v1/user/notifcation/:id
```

Response:

```
{
  "success": true
}
```

Ticket Endpoints

Create Ticket

```
POST /api/v1/ticket/create
```

Requires: `issue::create` permission

Request Body:

```
{
  "name": "string",
  "company": "company_id or object",
  "detail": "object",
  "title": "string",
  "priority": "low|medium|high|critical",
  "email": "string",
  "engineer": {
    "id": "user_id",
    "name": "string"
  },
  "type":
"support|bug|feature|incident|service|maintenance|access|feedback",
  "createdBy": {
    "id": "user_id",
```

```
    "name": "string",
    "role": "string",
    "email": "string"
  }
}
```

Response:

```
{
  "message": "Ticket created correctly",
  "success": true,
  "id": "ticket_id"
}
```

Create Ticket (Public)

```
POST /api/v1/ticket/public/create
```

No authentication required.

Request Body: Same as `/api/v1/ticket/create`

Response:

```
{
  "message": "Ticket created correctly",
  "success": true,
  "id": "ticket_id"
}
```

Get Ticket by ID

```
GET /api/v1/ticket/:id
```

Requires: `issue::read` permission

Response:

```
{
  "ticket": {
    "id": "uuid",
    "title": "string",
    "detail": "string",
    "priority": "string",
    "status": "needs_support|in_progress|in_review|hold|done",
    "isComplete": false,
    "type": "support",
    "email": "string",
    "name": "string",
    "Number": 1,
    "createdAt": "datetime",
    "updatedAt": "datetime",
    "client": {},
    "assignedTo": {
      "id": "uuid",
      "name": "string"
    },
    "comments": [],
    "TimeTracking": [],
    "files": []
  },
  "sucess": true
}
```

Get Open Tickets

```
GET /api/v1/tickets/open
```

Requires: `issue::read` permission

Response:

```
{
  "tickets": [],
  "sucess": true
}
```

Get All Tickets (Admin)

```
GET /api/v1/tickets/all
```

Response:

```
{
  "tickets": [],
  "sucess": true
}
```

Get User's Open Tickets

```
GET /api/v1/tickets/user/open
```

Response:

```
{
  "tickets": [],
  "sucess": true
}
```

Get Completed Tickets

```
GET /api/v1/tickets/completed
```

Response:

```
{
  "tickets": [],
  "sucess": true
}
```

Get Unassigned Tickets

```
GET /api/v1/tickets/unassigned
```

Response:

```
{
  "success": true,
  "tickets": []
}
```

Search Tickets

```
POST /api/v1/tickets/search
```

Requires: `issue::read` permission

Request Body:

```
{
  "query": "search_string"
}
```

Response:

```
{
  "tickets": [],
  "success": true
}
```

Update Ticket

```
PUT /api/v1/ticket/update
```

Requires: `issue::update` permission

Request Body:

```
{
  "id": "ticket_id",
  "note": "string",
  "detail": "object",
  "title": "string",
  "priority": "low|medium|high|critical",
  "status": "needs_support|in_progress|in_review|hold|done"
}
```

Response:

```
{
  "success": true
}
```

Transfer Ticket

```
POST /api/v1/ticket/transfer
```

Requires: `issue::transfer` permission

Request Body:

```
{
  "user": "user_id",
  "id": "ticket_id"
}
```

Set `user` to null to unassign.

Response:

```
{
  "success": true
}
```

Transfer Ticket to Client

```
POST /api/v1/ticket/transfer/client
```

Requires: `issue::transfer` permission

Request Body:

```
{
  "client": "client_id",
  "id": "ticket_id"
}
```

Response:

```
{
  "success": true
}
```

Comment on Ticket

```
POST /api/v1/ticket/comment
```

Requires: `issue::comment` permission

Request Body:

```
{
  "text": "string",
  "id": "ticket_id",
  "public": true
}
```

Response:

```
{
  "success": true
}
```

Delete Comment

```
POST /api/v1/ticket/comment/delete
```

Requires: `issue::comment` permission

Request Body:

```
{
  "id": "comment_id"
}
```

Response:

```
{
  "success": true
}
```

Update Ticket Status

```
PUT /api/v1/ticket/status/update
```

Requires: `issue::update` permission

Request Body:

```
{
  "status": true,
  "id": "ticket_id"
}
```

`status: true` marks as complete, `false` marks as incomplete.

Response:

```
{
  "success": true
}
```

Hide Ticket

```
PUT /api/v1/ticket/status/hide
```

Requires: `issue::update` permission

Request Body:

```
{
  "hidden": true,
  "id": "ticket_id"
}
```

Response:

```
{
  "success": true
}
```

Lock Ticket

```
PUT /api/v1/ticket/status/lock
```

Requires: `issue::update` permission

Request Body:

```
{
  "locked": true,
  "id": "ticket_id"
}
```

Response:

```
{
  "success": true
}
```

Delete Ticket

```
POST /api/v1/ticket/delete
```

Requires: `issue::delete` permission

Request Body:

```
{
  "id": "ticket_id"
}
```

Response:

```
{
  "success": true
}
```

Subscribe to Ticket

```
GET /api/v1/ticket/subscribe/:id
```

Requires: `issue::read` permission

Response:

```
{
  "success": true
}
```

Unsubscribe from Ticket

```
GET /api/v1/ticket/unsubscribe/:id
```

Requires: `issue::read` permission

Response:

```
{
  "success": true
}
```

Get External User's Open Tickets

```
GET /api/v1/tickets/user/open/external
```

Response:

```
{
  "tickets": [],
  "sucess": true
}
```

Get External User's Closed Tickets

```
GET /api/v1/tickets/user/closed/external
```

Response:

```
{
  "tickets": [],
  "sucess": true
}
```

Get All External User's Tickets

```
GET /api/v1/tickets/user/external
```

Requires: `issue::read` permission

Response:

```
{
  "tickets": [],
  "sucess": true
}
```

Client Management Endpoints

Create Client

```
POST /api/v1/client/create
```

Requires: `client::create` permission

Request Body:

```
{
  "name": "string",
  "email": "string",
  "number": "string",
  "contactName": "string"
}
```

Response:

```
{
  "success": true
}
```

Update Client

```
POST /api/v1/client/update
```

Requires: `client::update` permission

Request Body:

```
{
  "name": "string",
  "email": "string",
  "number": "string",
  "contactName": "string",
  "id": "client_id"
}
```

Response:

```
{
  "success": true
}
```

Get All Clients

```
GET /api/v1/clients/all
```

Requires: `client::read` permission

Response:

```
{
  "success": true,
  "clients": []
}
```

Delete Client

```
DELETE /api/v1/clients/:id/delete-client
```

Requires: `client::delete` permission

Response:

```
{
  "success": true
```

```
}
```

Notebook/Documentation Endpoints

Create Note

```
POST /api/v1/notebook/note/create
```

Requires: `document::create` permission

Request Body:

```
{
  "content": "string",
  "title": "string"
}
```

Response:

```
{
  "success": true,
  "id": "note_id"
}
```

Get All Notes

```
GET /api/v1/notebooks/all
```

Requires: `document::read` permission

Response:

```
{
  "success": true,
  "notebooks": []
}
```

Get Single Note

```
GET /api/v1/notebooks/note/:id
```

Requires: `document::read` permission

Response:

```
{
  "success": true,
  "note": {}
}
```

Update Note

```
PUT /api/v1/notebooks/note/:id/update
```

Requires: `document::update` permission

Request Body:

```
{
  "content": "string",
  "title": "string"
}
```

Response:

```
{
  "success": true
}
```

Delete Note

```
DELETE /api/v1/notebooks/note/:id
```

Requires: `document::delete` permission

Response:

```
{
  "success": true
}
```

Time Tracking Endpoints

Create Time Entry

```
POST /api/v1/time/new
```

Request Body:

```
{
  "time": 60,
  "ticket": "ticket_id",
  "title": "string",
  "user": "user_id"
}
```

Response:

```
{
  "success": true
}
```

Email Queue Endpoints

Create Email Queue

```
POST /api/v1/email-queue/create
```

Request Body:

```
{
  "name": "string",
```

```
  "username": "string",
  "password": "string",
  "hostname": "string",
  "tls": true,
  "serviceType": "gmail|other",
  "clientId": "string",
  "clientSecret": "string",
  "redirectUri": "string"
}
```

Response (Gmail):

```
{
  "success": true,
  "message": "Gmail imap provider created!",
  "authorizeUrl": "authorization_url"
}
```

Gmail OAuth Callback

```
GET /api/v1/email-queue/oauth/gmail
```

Query Parameters:

- `code` : OAuth code
- `mailboxId` : Mailbox ID

Response:

```
{
  "success": true,
  "message": "Mailbox updated!"
}
```

Get All Email Queues

```
GET /api/v1/email-queues/all
```

Response:

```
{
  "success": true,
  "queues": []
}
```

Delete Email Queue

```
DELETE /api/v1/email-queue/delete
```

Request Body:

```
{
  "id": "queue_id"
}
```

Response:

```
{
  "success": true
}
```

Webhook Endpoints

Create Webhook

```
POST /api/v1/webhook/create
```

Requires: `webhook::create` permission

Request Body:

```
{
  "name": "string",
  "url": "string",
  "type": "ticket_created|ticket_status_changed",
```

```
  "active": true,
  "secret": "string"
}
```

Response:

```
{
  "message": "Hook created!",
  "success": true
}
```

Get All Webhooks

```
GET /api/v1/webhooks/all
```

Requires: `webhook::read` permission

Response:

```
{
  "webhooks": [],
  "success": true
}
```

Delete Webhook

```
DELETE /api/v1/admin/webhook/:id/delete
```

Requires: `webhook::delete` permission

Response:

```
{
  "success": true
}
```

Configuration Endpoints

Check Authentication Config

```
GET /api/v1/config/authentication/check
```

Response:

```
{
  "success": true,
  "sso": false,
  "provider": ""
}
```

Update OIDC Provider

```
POST /api/v1/config/authentication/oidc/update
```

Request Body:

```
{
  "clientId": "string",
  "clientSecret": "string",
  "redirectUri": "string",
  "issuer": "string",
  "jwtSecret": "string"
}
```

Response:

```
{
  "success": true,
  "message": "OIDC config Provider updated!"
}
```

Update OAuth Provider

```
POST /api/v1/config/authentication/oauth/update
```

Request Body:

```
{
  "name": "string",
  "clientId": "string",
  "clientSecret": "string",
  "redirectUri": "string",
  "tenantId": "string",
  "issuer": "string",
  "jwtSecret": "string"
}
```

Response:

```
{
  "success": true,
  "message": "SSO Provider updated!"
}
```

Delete SSO Provider

```
DELETE /api/v1/config/authentication
```

Response:

```
{
  "success": true,
  "message": "SSO Provider deleted!"
}
```

Get Email Config

```
GET /api/v1/config/email
```

Response:

```
{
  "success": true,
  "active": false,
  "email": {}
}
```

Update Email Config

```
PUT /api/v1/config/email
```

Request Body:

```
{
  "host": "string",
  "active": true,
  "port": "587",
  "reply": "noreply@example.com",
  "username": "string",
  "password": "string",
  "serviceType": "gmail|other",
  "clientId": "string",
  "clientSecret": "string",
  "redirectUri": "string"
}
```

Response:

```
{
  "success": true,
  "message": "SSO Provider updated!"
}
```

Gmail OAuth Callback (Config)

```
GET /api/v1/config/email/oauth/gmail
```

Query Parameters:

- `code` : OAuth code

Response:

```
{
  "success": true,
  "message": "SSO Provider updated!"
}
```

Delete Email Config

```
DELETE /api/v1/config/email
```

Response:

```
{
  "success": true,
  "message": "Email settings deleted!"
}
```

Toggle Roles

```
PATCH /api/v1/config/toggle-roles
```

Requires: Admin access + `settings::manage` permission

Request Body:

```
{
  "isActive": true
}
```

Response:

```
{
  "success": true,
  "message": "Roles updated!"
}
```

Role Management Endpoints

Create Role

```
POST /api/v1/role/create
```

Requires: `role::create` permission

Request Body:

```
{
  "name": "string",
  "description": "string",
  "permissions": ["issue::create", "issue::read"],
  "isDefault": false
}
```

Response:

```
{
  "message": "Role created!",
  "success": true
}
```

Get All Roles

```
GET /api/v1/roles/all
```

Requires: `role::read` permission

Response:

```
{
  "roles": [],
  "success": true,
  "roles_active": false
}
```

Get Role by ID

```
GET /api/v1/role/:id
```

Requires: `role::read` permission

Response:

```
{
  "role": {},
  "success": true
}
```

Update Role

```
PUT /api/v1/role/:id/update
```

Requires: `role::update` permission

Request Body:

```
{
  "name": "string",
  "description": "string",
  "permissions": [],
  "isDefault": false,
  "users": ["user_id"]
}
```

Response:

```
{
  "role": {},
  "success": true
}
```

Delete Role

```
DELETE /api/v1/role/:id/delete
```

Requires: `role::delete` permission

Response:

```
{
  "success": true
}
```

Assign Role to User

```
POST /api/v1/role/assign
```

Requires: `role::update` permission

Request Body:

```
{
  "userId": "user_id",
  "roleId": "role_id"
}
```

Response:

```
{
  "user": {},
  "success": true
}
```

Remove Role from User

```
POST /api/v1/role/remove
```

Request Body:

```
{
  "userId": "user_id",
  "roleId": "role_id"
}
```

Response:

```
{
  "user": {},
  "success": true
}
```

Data/Statistics Endpoints

Get Total Ticket Count

```
GET /api/v1/data/tickets/all
```

Requires: `issue::read` permission

Response:

```
{
  "count": 100
}
```

Get Completed Ticket Count

```
GET /api/v1/data/tickets/completed
```

Requires: `issue::read` permission

Response:

```
{
  "count": 50
}
```

Get Open Ticket Count

```
GET /api/v1/data/tickets/open
```

Requires: `issue::read` permission

Response:

```
{
  "count": 50
}
```

Get Unassigned Ticket Count

```
GET /api/v1/data/tickets/unassigned
```

Requires: `issue::read` permission

Response:

```
{
  "count": 10
}
```

Get Logs

```
GET /api/v1/data/logs
```

Response:

```
{
  "logs": "log_content"
}
```

Storage Endpoints

Upload Ticket Attachment

```
POST /api/v1/storage/ticket/:id/upload/single
```

Content-Type: `multipart/form-data`

Form Data:

- `file` : The file to upload
- `user` : User ID

Response:

```
{
  "success": true
}
```

Permission System

Issue Permissions

- `issue::create`
- `issue::read`
- `issue::write`
- `issue::update`
- `issue::delete`
- `issue::assign`
- `issue::transfer`
- `issue::comment`

User Permissions

- `user::create`
- `user::read`
- `user::update`
- `user::delete`
- `user::manage`

Role Permissions

- `role::create`
- `role::read`
- `role::update`
- `role::delete`
- `role::manage`

Client Permissions

- `client::create`
- `client::read`
- `client::update`
- `client::delete`
- `client::manage`

Webhook Permissions

- `webhook::create`
- `webhook::read`
- `webhook::update`
- `webhook::delete`

Document Permissions

- `document::create`
- `document::read`
- `document::update`
- `document::delete`
- `document::manage`

System Permissions

- `settings::view`
- `settings::manage`
- `webhook::manage`
- `integration::manage`
- `email_template::manage`

Data Models

Ticket Model

- `id` : UUID
- `Number` : Auto-incrementing integer
- `title` : String
- `detail` : String (JSON)
- `note` : String
- `priority` : String (low, medium, high, critical)
- `status` : Enum (needs_support, in_progress, in_review, hold, done)
- `type` : Enum (bug, feature, support, incident, service, maintenance, access, feedback)
- `email` : String (requester email)
- `name` : String (requester name)
- `isComplete` : Boolean
- `hidden` : Boolean
- `locked` : Boolean
- `fromImap` : Boolean
- `following` : JSON array of user_ids
- `linked` : JSON array of linked tickets
- `createdBy` : JSON object
- `userId` : Foreign key to User (assignee)
- `clientId` : Foreign key to Client
- `teamId` : Foreign key to Team
- `createdAt` : DateTime
- `updatedAt` : DateTime

User Model

- `id` : UUID
- `name` : String

- `email` : String (unique)
- `password` : String (bcrypt hashed)
- `isAdmin` : Boolean
- `language` : String (default: "en")
- `external_user` : Boolean
- `firstLogin` : Boolean
- `notify_ticket_created` : Boolean
- `notify_ticket_status_changed` : Boolean
- `notify_ticket_comments` : Boolean
- `notify_ticket_assigned` : Boolean
- `out_of_office` : Boolean
- `out_of_office_message` : String
- `out_of_office_start` : DateTime
- `out_of_office_end` : DateTime
- `teamId` : Foreign key to Team
- `createdAt` : DateTime
- `updatedAt` : DateTime

Client Model

- `id` : UUID
- `name` : String
- `email` : String (unique)
- `contactName` : String
- `number` : String
- `notes` : String
- `active` : Boolean
- `createdAt` : DateTime
- `updatedAt` : DateTime

Comment Model

- `id` : UUID
- `text` : String
- `public` : Boolean
- `reply` : Boolean
- `replyEmail` : String
- `edited` : Boolean

- `editedAt` : DateTime
- `previous` : String
- `userId` : Foreign key to User
- `ticketId` : Foreign key to Ticket
- `createdAt` : DateTime

Webhook Model

- `id` : UUID
- `name` : String
- `url` : String
- `type` : Enum (ticket_created, ticket_status_changed)
- `active` : Boolean
- `secret` : String
- `createdBy` : String (user_id)
- `createdAt` : DateTime
- `updatedAt` : DateTime

Webhook Payloads

Ticket Created

```
{
  "event": "ticket_created",
  "id": "ticket_id",
  "title": "string",
  "priority": "string",
  "email": "string",
  "name": "string",
  "type": "string",
  "createdBy": {},
  "assignedTo": {},
  "client": {}
}
```

Ticket Status Changed

Sent to configured webhook URLs when ticket status is updated.

Notification System

Peppermint has built-in email notifications for:

- Ticket creation
- Ticket assignment
- Ticket comments
- Ticket status changes

Users can control email notifications via `/api/v1/auth/profile/notifcations/emails`.

Webhooks can be configured for external notification delivery (including Discord integration).

Important Notes

1. Most endpoints require authentication via JWT bearer token
2. Permission-based access control is enforced on most endpoints
3. Session tokens expire after 8 hours
4. Passwords are bcrypt hashed
5. The API uses Prisma ORM for PostgreSQL database access
6. Ticket numbers auto-increment
7. Soft deletes used for some entities (hidden flag)
8. File uploads handled by multer
9. OAuth/OIDC support for SSO
10. Gmail OAuth supported for email sending
    ouroboroz@OUROBOROZ-SERVER:~/Projects/AI/LLMs/Ha